1.Write a python program to solve 8-Puzzle problem

```
from collections import deque

goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def solve(start):
    queue = deque()
    queue.append((start, []))

    visited = set()
    visited.add(start)

    while queue:
        state, path = queue.popleft()

        if state == goal:
            return path + [state]

        zero = state.index(0)
        row = zero // 3
        col = zero % 3
```

```
moves = [
    (-1, 0),
    (1, 0),
    (0, -1),
    (0, 1)
]

for dr, dc in moves:
    new_row = row + dr
    new_col = col + dc

    if 0 <= new_row < 3 and 0 <= new_col < 3:

        new_zero = new_row * 3 + new_col

        new_state = list(state)

        new_state[zero] = new_state[new_zero]
        new_state[new_zero] = 0

        new_state = tuple(new_state)

        if new_state not in visited:
```

```
                visited.add(new_state)
                queue.append(
                    (new_state, path + [state])
                )

    return None


def display(state):
    print(state[0], state[1], state[2])
    print(state[3], state[4], state[5])
    print(state[6], state[7], state[8])
    print()


print("Enter 9 numbers (0 represents blank):")

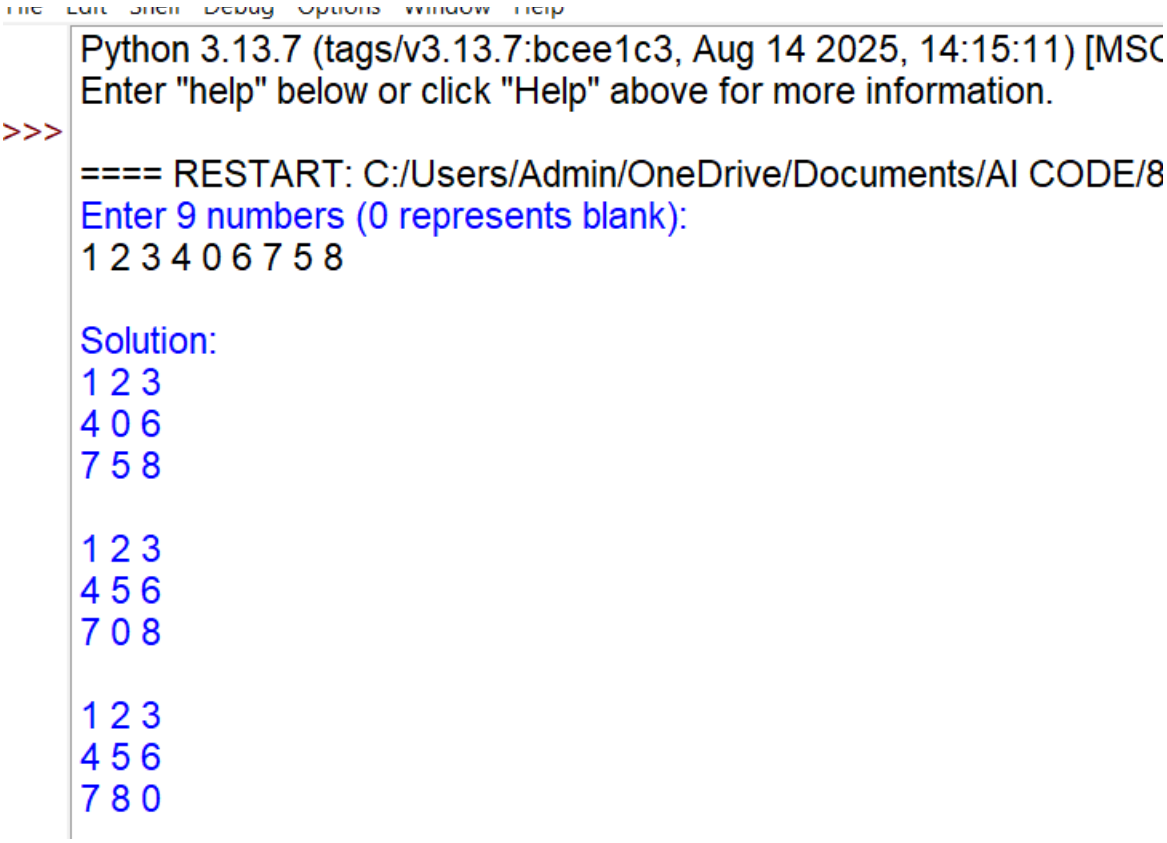
start = tuple(map(int, input().split()))

solution = solve(start)

if solution:
    print("\nSolution:")
```

```
    for state in solution:
        display(state)
else:
    print("No solution exists.")
```

OUTPUT:



2. Write a python program to solve 8-Queen problem

```
N = 8

board = [-1] * N

def is_safe(row, col):
    for i in range(row):
        if board[i] == col:
            return False

        if abs(board[i] - col) == abs(i - row):
            return False

    return True



def solve(row):
    if row == N:
        print_board()
        return True

    for col in range(N):
```

```
        if is_safe(row, col):
            board[row] = col

            if solve(row + 1):
                return True

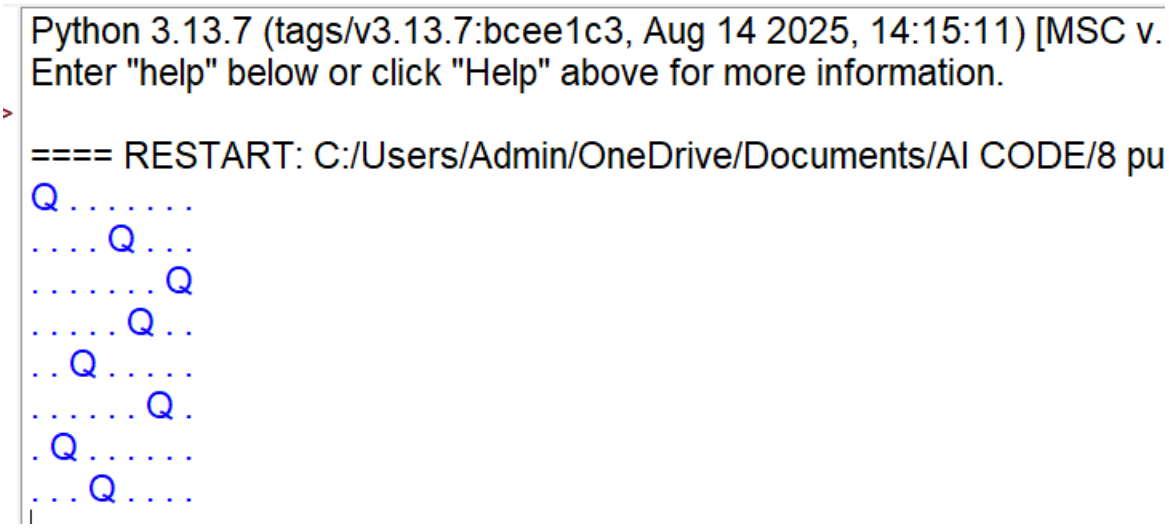
            board[row] = -1

    return False


def print_board():
    for row in range(N):
        for col in range(N):
            if board[row] == col:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()


solve(0)
```

OUTPUT



3. Write a python program for water jug problem

```
from collections import deque

def water_jug(jug1, jug2, target):
    queue = deque()
    visited = set()

    queue.append((0, 0, []))

    while queue:
        a, b, path = queue.popleft()
```

```
if (a, b) in visited:
    continue

visited.add((a, b))

if a == target or b == target:
    for step in path:
        print(step)
    print("Goal reached!")
    return

states = [
    (jug1, b, "Fill Jug 1"),
    (a, jug2, "Fill Jug 2"),
    (0, b, "Empty Jug 1"),
    (a, 0, "Empty Jug 2")
]

pour = min(a, jug2 - b)
states.append((a - pour, b + pour, "Pour Jug 1 into Jug 2"))

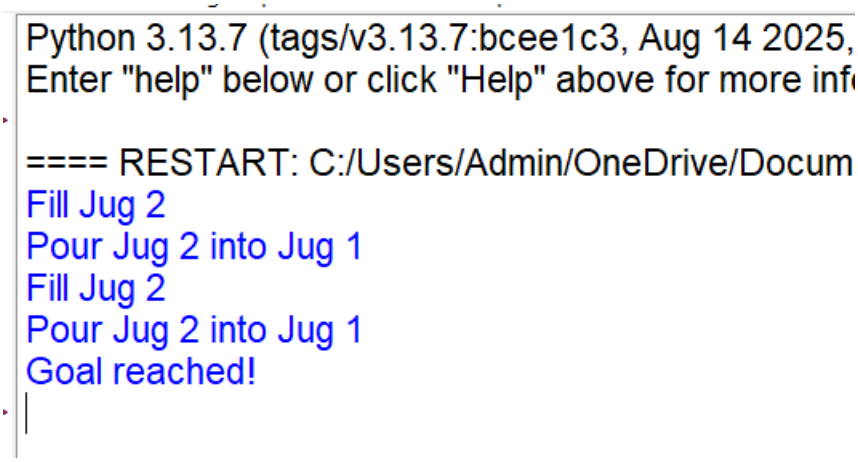
pour = min(b, jug1 - a)
```

```
        states.append((a + pour, b - pour, "Pour Jug 2 into Jug 1"))

        for new_a, new_b, action in states:
            if (new_a, new_b) not in visited:
                queue.append((new_a, new_b, path + [action]))

water_jug(4, 3, 2)
```

OUTPUT



4. Write a python program for Cript-Arithmetic problem

```
from itertools import permutations

letters = "SENDMORY"

for p in permutations(range(10), len(letters)):
    S, E, N, D, M, O, R, Y = p

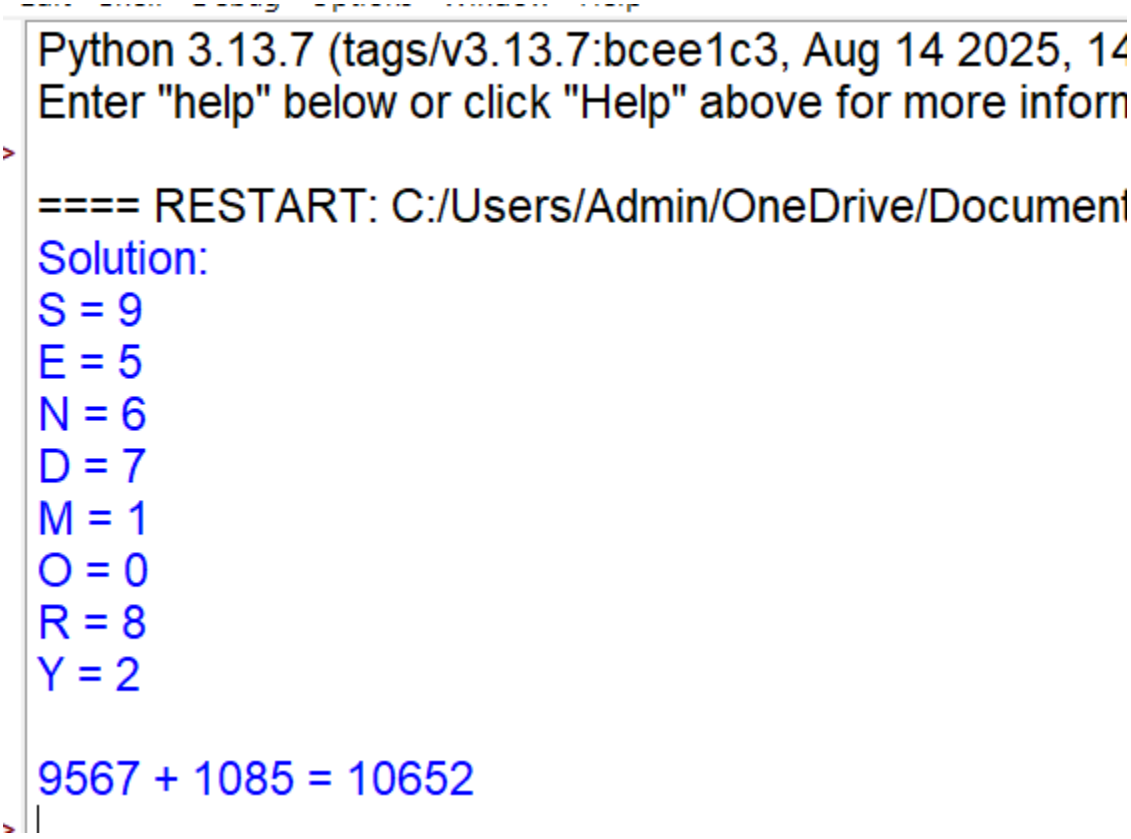
    if S == 0 or M == 0:
        continue

    SEND = 1000*S + 100*E + 10*N + D
    MORE = 1000*M + 100*O + 10*R + E
    MONEY = 10000*M + 1000*O + 100*N + 10*E + Y

    if SEND + MORE == MONEY:
        print("Solution:")
        print("S =", S)
        print("E =", E)
        print("N =", N)
        print("D =", D)
```

```
    print("M =", M)
    print("O =", O)
    print("R =", R)
    print("Y =", Y)
    print()
    print(SEND, "+", MORE, "=", MONEY)
    break
```

OUTPUT



5. Write a python program for Missionaries Cannibal Problem

```
from collections import deque

def is_valid(m_left, c_left, m_right, c_right):
    if m_left < 0 or c_left < 0 or m_right < 0 or c_right < 0:
        return False

    if m_left > 0 and m_left < c_left:
        return False

    if m_right > 0 and m_right < c_right:
        return False

    return True



def solve():
    start = (3, 3, 0)
    goal = (0, 0, 1)

    queue = deque()
```

```
queue.append((start, []))

visited = set()

moves = [
    (1, 0),
    (2, 0),
    (0, 1),
    (0, 2),
    (1, 1)
]

while queue:
    state, path = queue.popleft()

    if state in visited:
        continue

    visited.add(state)

    m_left, c_left, boat = state
    m_right = 3 - m_left
    c_right = 3 - c_left
```

```
        if state == goal:
            for step in path:
                print(step)
            print("Goal reached!")
            return

        for m, c in moves:
            if boat == 0:
                new_state = (m_left - m, c_left - c, 1)
                side = "Left to Right"
            else:
                new_state = (m_left + m, c_left + c, 0)
                side = "Right to Left"

            nm, nc, nb = new_state
            nmr = 3 - nm
            ncr = 3 - nc

            if is_valid(nm, nc, nmr, ncr):
                if new_state not in visited:
                    action = "Move " + str(m) + " Missionary(s) and " + str(c) +
" Cannibal(s) " + side
```

```
queue.append((new_state, path + [action]))
```

```
solve()
```

OUTPUT

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64
Enter "help" below or click "Help" above for more information.

==== RESTART: C:/Users/Admin/OneDrive/Documents/AI CODE/8 puzzle prog
Move 0 Missionary(s) and 2 Cannibal(s) Left to Right
Move 0 Missionary(s) and 1 Cannibal(s) Right to Left
Move 0 Missionary(s) and 2 Cannibal(s) Left to Right
Move 0 Missionary(s) and 1 Cannibal(s) Right to Left
Move 2 Missionary(s) and 0 Cannibal(s) Left to Right
Move 1 Missionary(s) and 1 Cannibal(s) Right to Left
Move 2 Missionary(s) and 0 Cannibal(s) Left to Right
Move 0 Missionary(s) and 1 Cannibal(s) Right to Left
Move 0 Missionary(s) and 2 Cannibal(s) Left to Right
Move 1 Missionary(s) and 0 Cannibal(s) Right to Left
Move 1 Missionary(s) and 1 Cannibal(s) Left to Right
Goal reached!
```